

1 Áttekintés

A feladat egy $n \times n$ méretű mátrixban kell megtalálni a maximális összegű útvonalat a $(1, 1)$ cellából a (n, n) cellába. Az engedélyezett lépések: jobbra, lefelé, és pontosan egy átló menti lépés. Ez utóbbi azt jelenti, hogy a sor- és oszlopindexek egyaránt változnak egyetlen lépésben.

A triviális dinamikus programozás nem működik közvetlenül, mert egy átló menti lépés során „visszafelé” kell haladnunk a szokásos feldolgozási sorrendben. A megoldás kulcsa, hogy három DP-állapotot használunk, amelyek mindegyike egy-egy jól meghatározott útvonal-típust ír le.

2 DP-állapotok

Jelölje $m[i][j]$ a mátrix i -edik sorának j -edik oszlopában tárolt értéket. Az alábbi állapotokat definiáljuk az (i, j) cellára:

- **dp0[i][j]**: A maximális összeg a $(1, 1)$ cellából az (i, j) cellába olyan útvonalon, amely csak jobbra és lefelé lépéseket tartalmaz.
- **dp1[i][j]**: A maximális összeg olyan útvonalon, amely eddig csak jobbra/lefelé lépéseket tartalmazott, és most egy $\nwarrow$ irányú átló menti lépéssel érkezett az (i, j) cellába az $(i - 1, j + 1)$ cellából.
- **dp2[i][j]**: A maximális összeg olyan útvonalon, amely eddig csak jobbra/lefelé lépéseket tartalmazott, és most egy $\searrow$ irányú átló menti lépéssel érkezett az (i, j) cellába az $(i + 1, j - 1)$ cellából.
- **dp[i][j]**: Az (i, j) cellában elérhető maximális összeg, azaz $\max(\text{dp0}, \text{dp1}, \text{dp2})$.

A kezdőérték minden állapotnál $m[1][1]$, hiszen a kiindulócellából még nem tettünk lépést.

3 Átmenetek

Az átlós lépések miatt a szokásos sorfolytonos feldolgozás nem elegendő. Az algoritmus az *antiátlók* mentén halad (minden (i, j) párra az $i + j$ konstans), így biztosítva, hogy egy cella összes lehetséges elődcellája már feldolgozott, mire az adott cellát számoljuk.

3.1 dp0 frissítése (csak szigorúan monoton lépések)

$$\text{dp0}[i][j] = \max(\text{dp}[i - 1][j], \text{dp}[i][j - 1]) + m[i][j]$$

Csak a felülről és balról érkező útvonalakat vizsgáljuk, amelyek már tartalmazhatnak egy korábbi átlós lépést is.

3.2 dp1 frissítése ($\nwarrow$ átló)

$$dp1[i][j] = \max(dp1[i-1][j+1], dp0[i-1][j+1]) + m[i][j]$$

Az (i, j) cellába $\nwarrow$ irányban az $(i-1, j+1)$ cellából érkezhethetünk. Ha eddig még nem volt átlós lépés, akkor a dp0-állapotot használjuk; ha már volt egy korábbi $\nwarrow$ átlós lépés, akkor a dp1-et.

3.3 dp2 frissítése ($\swarrow$ átló)

$$dp2[i][j] = \max(dp2[i+1][j-1], dp0[i+1][j-1]) + m[i][j]$$

Az (i, j) cellába $\swarrow$ irányban az $(i+1, j-1)$ cellából érkezhethetünk. Az átmenet hasonló a dp1-hez: ha még nem volt átlós lépés, dp0-ból, különben dp2-ből jövünk.

3.4 dp összefűzése

$$dp[i][j] = \max(dp0[i][j], dp1[i][j], dp2[i][j])$$

Ez az egyesített állapot reprezentálja az (i, j) cellába vezető összes lehetséges útvonalat.

4 Algoritmus

```
for (int antidiag = 3; antidiag <= 2 * n; antidiag++) {
    // 1. dp0: csak le/fel lépések
    for (int i = 1; i <= n; i++) {
        int j = antidiag - i;
        if (j < 1 || j > n) continue;
        dp0[i][j] = max({dp0[i][j], dp[i-1][j] + m[i][j], dp
            [i][j-1] + m[i][j]});
    }
    // 2. dp1: atlo fel-le (jobb-ra-le)
    for (int i = 1; i <= n; i++) {
        int j = antidiag - i;
        if (j < 1 || j > n) continue;
        dp1[i][j] = max({dp1[i][j], dp1[i-1][j+1] + m[i][j],
            dp0[i-1][j+1] + m[i][j]});
    }
    // 3. dp2: atlo le-fel (le-jobb-ra)
    for (int i = n; i > 0; i--) {
        int j = antidiag - i;
        if (j < 1 || j > n) continue;
        dp2[i][j] = max({dp2[i][j], dp2[i+1][j-1] + m[i][j],
            dp0[i+1][j-1] + m[i][j]});
    }
    // 4. osszefuzes
    for (int i = 1; i <= n; i++) {
```

```

        int j = antidiag - i;
        if (j < 1 || j > n) continue;
        dp[i][j] = max({dp[i][j], dp0[i][j], dp1[i][j], dp2[
            i][j]});
    }
}

```

Az antiátló-feldolgozás biztosítja, hogy minden szükséges elődcellát már korábban kiszámoltunk. Az iteráció $O(n^2)$ cellát dolgoz fel, és minden cellánál $O(1)$ műveletet végzünk.

5 Idő- és tárkomplexitás

- **Időkomplexitás:** $O(n^2)$, mivel minden cella pontosan egyszer kerül feldolgozásra minden átmenet-típusra.
- **Tárkomplexitás:** $O(n^2)$, négy $n \times n$ méretű DP-táblázatot tárolunk.

6 Kulcs megfigyelések

1. Az átló menti lépés „visszafelé” halad a szokásos mátrix-bejárási sorrendben, ezért az antiátló szerinti feldolgozás elengedhetetlen.
2. Három DP-állapot elegendő, mert a feladat csak egyetlen átlós lépés engedélyezését kívánja meg, és az útvonalak szerkezete jól karakterizálható a $\nwarrow$ és $\swarrow$ irányokkal.
3. A dp0-állapot „hídként” szolgál: lehetővé teszi, hogy az első átlós lépés után újra „tisztá” jobbra/lefelé lépéseket tartalmazó útvonalat folytassunk.

A válasz a (n, n) cella dp-értéke: $\text{dp}[n][n]$.